



Discipline : *Algorithmique 2*

Semestre : *2*

Niveau d'étude : *Licence 1 Génie informatique*

Année académique: *2023/2024*

Chapitre 3 : Les structures répétitives (Les boucles)

PRÉREQUIS A VERIFIER

- ☐ *Les bases de l'algorithmique*
- ☐ *La structure séquentielle*
- ☐ *Les structures de contrôle conditionnelles*

Introduction:

PROBLÈME

*Ecrire un algorithme qui permet d'afficher 1000 fois à l'écran la phrase suivante « **Je me tais dès que le Professeur entre en classe** ».*

1. Problématique

Comment réaliser une exécution en boucle d'une ou de plusieurs instructions dans un algorithme ?



*En algorithmie l'une des solutions consiste à utiliser une forme particulière de structures connues sous le nom de structures de contrôle répétitives ou **boucles**.*

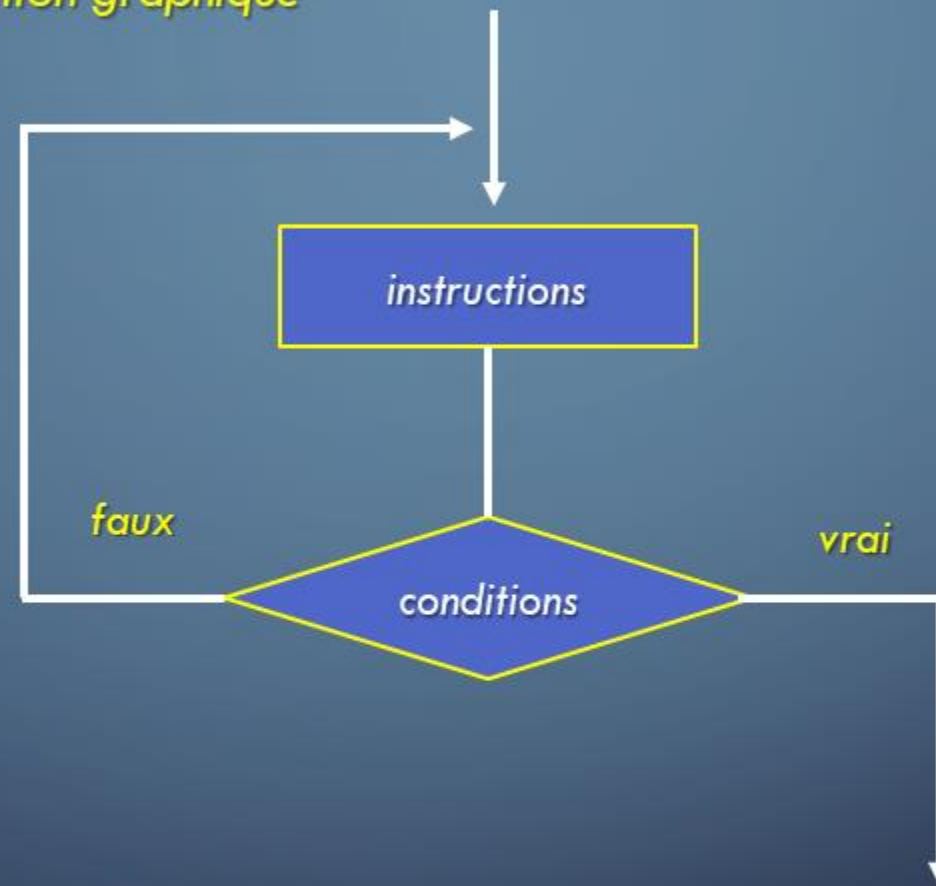
2. Typologie des boucles

On distingue essentiellement les structures

- ☐ *Répéter jusqu'à*
- ☐ *Pour*
- ☐ *Tant que*

2.1 La structure « repeter jusqu'à »

2.1.1 Représentation graphique



Commentaire : Dans une telle structure le bloc d'instructions est exécuté **au moins une fois** car le test s'effectue à la fin du premier passage dans la boucle. Ainsi l'enclenchement du prochain passage dans la boucle dépend du résultat obtenu à l'issue du test en cours.

Un résultat du test correspondant à « **vrai** » entraîne systématiquement l'arrêt de la boucle tandis que le résultat « **faux** » aboutit à un autre passage.

2.1.2 En pseudo code

Repeter

ACTIONS

Jusqu'à (**CONDITIONS**)

NB : Ici on répète l'exécution des instructions jusqu'à ce que la condition soit remplie

Exemple:

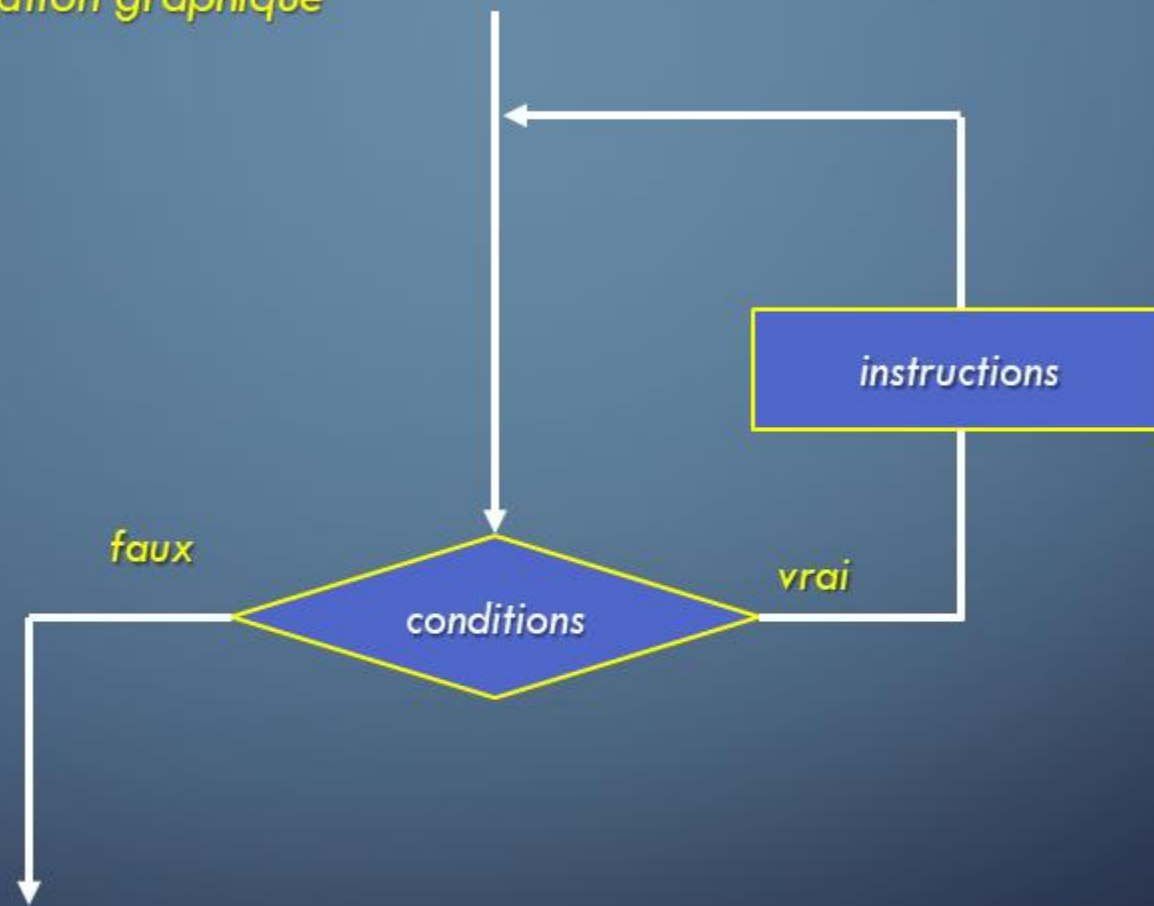
dans l'algorithme ci-dessous on continue de demander à l'utilisateur de saisir un nombre jusqu'à ce qu'il rentre la valeur 3

```
Algo          application1;  
  
Variable  
  
    N: entier ;  
  
Debut  
  
    Repeter  
        Ecrire ( `` saisir un entier ``);  
        Lire ( N );  
        Jusqu'à ( N = 3)  
    Fin.
```

RMQ : Ici la valeur 3 est appelée sentinelle car une fois saisie permet une sortie définitive dans la boucle .

2.2 La structure « Tant que »

2.2.1 Représentation graphique



Commentaire : Contrairement à la structure « repeter jusqu'à » le bloc d'instructions ici peut ne pas être exécuté car le test s'effectue avant l'entrée dans la boucle. Autrement dit au cas où la première tentative de pénétration dans la boucle se solde par un échec (résultat du test correspondant à « **faux** »), les instructions à l'intérieur de la boucle ne feront l'objet d'aucune exécution.

2.2.2 En pseudo code

Tant que (**CONDITIONS**) faire

ACTIONS

Fin Tant que

RMQ : Ici on part exécuter les instructions tant que la condition est remplie

Exemple:

dans l'algorithme ci-dessous on continue de demander à un utilisateur de saisir un nombre jusqu'à ce qu'il rentre la valeur 3

Algo application1 ;

Variable

N: entier ;

$N \leftarrow 0$;

Debut

Tant que (N < > 3) faire

Ecrire (`` saisir un entier ``);

Lire (N);

Fin tant que

Fin.

RMQ : Ici on a initialisé la variable de test à 0 en vue de garantir au moins un passage dans la boucle .

2.3. Moyens de contrôle des boucles

*Dans le jargon de la programmation informatique une répétition incontrôlable est qualifiée de **boucle infinie**.*

Cependant des moyens existent pour maîtriser, surveiller ou contrôler l'exécution en boucle d'une ou de plusieurs instructions. Parmi les instruments de contrôle des boucles nous pouvons citer:

- ✓ Les **sentinelles**,
- ✓ Les **compteurs** et
- ✓ Les **accumulateurs**.

2.3. 1 Une sentinelle

Une sentinelle comme nous venons de le voir est une valeur qui une fois saisie entraine une sortie dans la boucle

2.3. 2 Le compteur

Un compteur est une variable arbitraire destinée à s'occuper d'un dénombrement durant l'exécution d'un programme. Le processus de comptage se fait grâce au procédé d'incrémentation soit :

$$\text{Nom_Compteur} \leftarrow \text{Nom_Compteur} + \text{Pas}$$

Nota 1 : L'utilisation d'un compteur n'a de sens que lorsque la variable compteur est *initialisée* en amont, assujettie progressivement à des *incrémentations* et à laquelle un *plafond* est ou limite est fixée

Nota 2 : Les compteurs sont toujours de type entier. De plus le pas en question est aussi un entier. Si ce dernier est positif on parlera d'une *incrément* et au cas contraire(c'est-à-dire négatif) il s'agira d'une *décrément*.

2.3. 3 L'accumulateur

Un Accumulateur est une variable arbitraire destinée à engranger des données numériques durant l'exécution d'un algorithme ou d'un programme. Le processus d'accumulation des données est formalisée comme suit:

$$\text{Nom_Accumulateur} \leftarrow \text{Nom_Accumulateur} + \text{VALEUR}$$

Nota 1 : L'utilisation de accumulateur n'a de sens que lorsque l'accumulateur détient une valeur initiale.

RMQ : L'accumulateur au même titre que les valeurs qu'il engrange progressivement peut être de divers types (entier, réel etc.). De plus la valeur ajoutée ou défalquée peut varier d'une opération à une autre contrairement aux compteurs qui engrange que des valeurs constantes (plus connues sous le nom de PAS)

2.4 boucle Pour

Pour i allant de 1 à n faire

ACTIONS

Fin Pour;

Nota 1: Avec la boucle Pour l'initialisation du compteur (le plus souvent identifié par i), son incrémentation et la définition du niveau d'arrêt de la boucle se fait en un seul coup.

Cette forme de structure ne peut être déployée que si et seulement si à priori on a une idée du nombre de répétition à effectuer avant le lancement de la boucle

Nota 2: Le concepteur de l'algorithme est tenu de préciser le pas au cas où il est différent de 1

Application 1: Proposer un algorithme qui permet de résoudre le problème de la partie introduction de ce cours.

Vous résoudrez le problème de trois manières différentes c'est-à-dire en utilisant des structures itératives différentes.

Travaux dirigés

Exercice 1: Ecrire un algorithme qui exige à l'utilisateur de saisir un nombre entier strictement positif et qui affiche à l'écran le nombre saisi.

Exercice 2: Ecrire un algorithme qui exige à l'utilisateur de saisir un nombre entier strictement positif et qui affiche à l'écran les 10 premiers nombres pairs supérieurs à la valeur saisie.

Exercice 3: Ecrire un algorithme qui permet de calculer et d'afficher la somme des nombres entiers compris entre deux entiers saisis au clavier par l'utilisateur.

NB: Veillez à ce que les deux valeurs saisies soient différentes

Exercice 4: Ecrire un algorithme qui permet de calculer et d'afficher la somme des nombres entiers pairs compris entre 1 et 2000.

Exercice 5: Ecrire un algorithme qui permet de calculer et d'afficher la somme des nombres entiers impairs compris entre 0 et 5000.

Exercice 6: Ecrire un algorithme qui demande à l'utilisateur de saisir deux entiers différents et qui calcule et affiche la somme et le produit des nombres pairs compris en ces deux nombres.

Exercice 7: Ecrire un algorithme qui demande à l'utilisateur de saisir un nombre et qui calcule et affiche la factorielle d'un nombre saisi au clavier comme le font les calculatrices.

Exercice 8: Ecrire un algorithme qui permet de calculer et d'afficher la somme des nombres divisibles par 3 compris entre 1 et 2500.

Exercice 9: Ecrire un algorithme qui permet de calculer et d'afficher la somme des nombres premiers compris entre deux entiers saisis au clavier par l'utilisateur.

Exercice 10: Sachant que le mot de passe d'un utilisateur est XXLL2023, on vous demande de proposer un algorithme qui permet de gérer le système d'authentification.

NB: l'utilisateur n'a droit qu'à trois tentatives.